Filter Contents

In the previous section, we explored how to use redirection to send the output of one program into another for further processing. Now, let's talk about reading files directly from the command line, without needing to open a text editor.

There are two powerful tools for this - `more` and `less`. These are known as pagers, and they allow you to view the contents of a file interactively, one screen at a time. While both tools serve a similar purpose, they have some differences in functionality, which we'll touch on later.

Using `more` and `less`, you can easily scroll through large files, search for text, and navigate forward or backward without modifying the file itself. This is especially useful when you're working with large logs or text files that don't fit neatly into one screen.

The goal for this section is to learn how to filter content and handle the redirected output from previous commands. But before we dive into filtering, we need to become familiar with some essential tools and commands that are specifically designed to make filtering more efficient and powerful.

Before we start filtering the output of commands, let's explore a few foundational tools that will help you efficiently sift through and manipulate text. These tools are crucial when working with large amounts of data or when you need to automate tasks that involve searching, sorting, or processing information.

Let's look at some examples to understand how these tools work in practice.

More

```shellsession
athane@htb[/htb]$ cat /etc/passwd | more
```

The `/etc/passwd` file in Linux is like a phone directory for users on the system. It includes details such as the username, user ID, group ID, home directory, and the default shell they use.

After we read the content using `cat` and redirected it to `more`, the already mentioned `pager` opens, and we will automatically start at the beginning of the file.

```shellsession
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
<SNIP>
--More--
```

With the `[Q]` key, we can leave this `pager`. We will notice that the output remains in the terminal.

Less

If we now take a look at the tool `less`, we will notice on the man page that it contains many more features than `more`.

```shellsession
athane@htb[/htb]$ less /etc/passwd
```

The presentation is almost the same as with `more`.

```shellsession
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
<SNIP>
:
```

When closing `less` with the `[Q]` key, we will notice that the output we have seen, unlike `more`, does not remain in the terminal.

Head

Sometimes we will only be interested in specific issues either at the beginning of the file or the end. If we only want to get the `first` lines of the file, we can use the tool `head`. By default, `head` prints the first ten lines of the given file or input, if not specified otherwise.

```shellsession
athane@htb[/htb]$ head /etc/passwd

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
```

Tail

If we only want to see the last parts of a file or results, we can use the counterpart of `head` called `tail`, which returns the `last` ten lines.

```shellsession
athane@htb[/htb]$ tail /etc/passwd
```

```
miredo:x:115:65534::/var/run/miredo:/usr/sbin/nologin
usbmux:x:116:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
rtkit:x:117:119:RealtimeKit,,,:/proc:/usr/sbin/nologin
nm-openvpn:x:118:120:NetworkManager
OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin
nm-openconnect:x:119:121:NetworkManager OpenConnect
plugin,,,:/var/lib/NetworkManager:/usr/sbin/nologin
pulse:x:120:122:PulseAudio daemon,,,:/var/run/pulse:/usr/sbin/nologin
beef-xss:x:121:124::/var/lib/beef-xss:/usr/sbin/nologin
lightdm:x:122:125:Light Display Manager:/var/lib/lightdm:/bin/false
do-agent:x:998:998::/home/do-agent:/bin/false
user6:x:1000:1000:,,,:/home/user6:/bin/bash
```

It would be highly beneficial to explore the available options these tools offer and experiment with them.

Sort

Depending on which results and files are dealt with, they are rarely sorted. Often it is necessary to sort the desired results alphabetically or numerically to get a better overview. For this, we can use a tool called `sort`.

shellsession

```
athane@htb[/htb]$ cat /etc/passwd | sort

_apt:x:104:65534::/nonexistent:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
```

```
cry0l1t3:x:1001:1001::/home/cry0l1t3:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
dnsmasq:x:107:65534:dnsmasq,,,:/var/lib/misc:/usr/sbin/nologin
dovecot:x:114:117:Dovecot mail server,,,:/usr/lib/dovecot:/usr/sbin/nologin
dovenull:x:115:118:Dovecot login user,,,:/nonexistent:/usr/sbin/nologin
ftp:x:113:65534::/srv/ftp:/usr/sbin/nologin
games:x:5:60:games:/usr/games:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
htb-student:x:1002:1002::/home/htb-student:/bin/bash
<SNIP>
```

As we can see now, the output no longer starts with root but is now sorted alphabetically.

Grep

In many cases, we will need to search for specific results that match patterns we define. One of the most commonly used tools for this purpose is grep, which provides a wide range of powerful features for pattern searching. For instance, we can use grep to search for users who have their default shell set to `/bin/bash`.

shellsession

```
athane@htb[/htb]$ cat /etc/passwd | grep "/bin/bash"

root:x:0:0:root:/root:/bin/bash
mrb3n:x:1000:1000:mrb3n:/home/mrb3n:/bin/bash
cry0l1t3:x:1001:1001::/home/cry0l1t3:/bin/bash
htb-student:x:1002:1002::/home/htb-student:/bin/bash
```

This is just one example of how grep can be applied to efficiently filter data based on predefined patterns. Another possibility is to exclude specific results. For this, the option " `-v` " is used with `grep`. In the next example, we exclude all users who have disabled the standard shell with the name " `/bin/false` " or " `/usr/bin/nologin` ".

```shellsession
athane@htb[/htb]$ cat /etc/passwd | grep -v "false\|nologin"

root:x:0:0:root:/root:/bin/bash
sync:x:4:65534:sync:/bin:/bin/sync
postgres:x:111:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
user6:x:1000:1000:,,,:/home/user6:/bin/bash
```

Cut

Specific results with different characters may be separated as delimiters. Here it is handy to know how to remove specific delimiters and show the words on a line in a specified position. One of the tools that can be used for this is `cut`. Therefore we use the option " `-d` " and set the delimiter to the colon character (`:`) and define with the option " `-f` " the position in the line we want to output.

```shellsession
athane@htb[/htb]$ cat /etc/passwd | grep -v "false\|nologin" | cut -d":" -f1

root
sync
postgres
```

```
mrb3n
cry0l1t3
htb-student
```

Tr

Another possibility to replace certain characters from a line with characters defined by us is the tool `tr`. As the first option, we define which character we want to replace, and as a second option, we define the character we want to replace it with. In the next example, we replace the colon character with space.

```shellsession
athane@htb[/htb]$ cat /etc/passwd | grep -v "false\|nologin" | tr ":" " "

root x 0 0 root /root /bin/bash
sync x 4 65534 sync /bin /bin/sync
postgres x 111 117 PostgreSQL administrator,,, /var/lib/postgresql /bin/bash
mrb3n x 1000 1000 mrb3n /home/mrb3n /bin/bash
cry0l1t3 x 1001 1001  /home/cry0l1t3 /bin/bash
htb-student x 1002 1002  /home/htb-student /bin/bash
```

Column

Since search results can often have an unclear representation, the tool `column` is well suited to display such results in tabular form using the "`-t`."

```shellsession
athane@htb[/htb]$ cat /etc/passwd | grep -v "false\|nologin" | tr ":" " " | column -t

root         x  0     0      root              /root               /bin/bash
sync         x  4     65534  sync              /bin                /bin/sync
postgres     x  111   117    PostgreSQL        administrator,,,
/var/lib/postgresql     /bin/bash
mrb3n        x  1000  1000   mrb3n             /home/mrb3n         /bin/bash
cry0l1t3     x  1001  1001   /home/cry0l1t3    /bin/bash
htb-student  x  1002  1002   /home/htb-student  /bin/bash
```

Awk

As we may have noticed, the line for the user " `postgres` " has one column too many. To keep it as simple as possible to sort out such results, the (`g`) `awk` programming is beneficial, which allows us to display the first (`$1`) and last (`$NF`) result of the line.

```shellsession
athane@htb[/htb]$ cat /etc/passwd | grep -v "false\|nologin" | tr ":" " " | awk '{print $1, $NF}'

root /bin/bash
sync /bin/sync
postgres /bin/bash
mrb3n /bin/bash
cry0l1t3 /bin/bash
htb-student /bin/bash
```

Sed

There will come moments when we want to change specific names in the whole file or standard input. One of the tools we can use for this is the stream editor called `sed`. One of the most common uses of this is substituting text. Here, `sed` looks for patterns we have defined in the form of regular expressions (regex) and replaces them with another pattern that we have also defined. Let us stick to the last results and say we want to replace the word "`bin`" with "`HTB`."

The "`s`" flag at the beginning stands for the substitute command. Then we specify the pattern we want to replace. After the slash (`/`), we enter the pattern we want to use as a replacement in the third position. Finally, we use the "`g`" flag, which stands for replacing all matches.

```shellsession
athane@htb[/htb]$ cat /etc/passwd | grep -v "false\|nologin" | tr ":" " " | awk '{print $1, $NF}' | sed 's/bin/HTB/g'

root /HTB/bash
sync /HTB/sync
postgres /HTB/bash
mrb3n /HTB/bash
cry0l1t3 /HTB/bash
htb-student /HTB/bash
```

Wc

Last but not least, it will often be useful to know how many successful matches we have. To avoid counting the lines or characters manually, we can use the tool `wc`. With the "`-l`" option, we specify that only the lines are counted.

```shellsession
athane@htb[/htb]$ cat /etc/passwd | grep -v "false\|nologin" | tr ":" " " | awk '{print $1, $NF}' | wc -l

6
```

Practice

Keep in mind that there are numerous other tools available that you can utilize and incorporate throughout your journey. It's highly recommended to explore alternative tools for specific tasks to broaden your skill set, as you may discover options that better suit your personal preferences and workflows. There are no rigid limitations, so feel free to explore different possibilities and take advantage of the wealth of resources shared within the community.

It may be a bit overwhelming at first to deal with so many different tools and their functions if we are not familiar with them. Take your time and experiment with the tools. Have a look at the man pages (`man <tool>`) or call the help for it (`<tool> -h` / `<tool> --help`). The best way to become familiar with all the tools is to practice. Try to use them as often as possible, and we will be able to filter many things intuitively after a short time.

Here are a few optional exercises we can use to improve our filtering skills and get more familiar with the terminal and the commands. The file we will need to work with is the `/etc/passwd` file

on our `target` and we can use any shown command above. Our goal is to filter and display only specific contents. Read the file and filter its contents in such a way that we see only:

1. A line with the username `cry0l1t3`.
2. The usernames.
3. The username `cry0l1t3` and his UID.
4. The username `cry0l1t3` and his UID separated by a comma (`,`).
5. The username `cry0l1t3`, his UID, and the set shell separated by a comma (`,`).
6. All usernames with their UID and set shells separated by a comma (`,`).
7. All usernames with their UID and set shells separated by a comma (`,`) and exclude the ones that contain `nologin` or `false`.
8. All usernames with their UID and set shells separated by a comma (`,`) and exclude the ones that contain `nologin` and count all lines of the filtered output.

Connect to HTB

Target(s)

Spawn the target system to get IPs and answer questions

Spawn the target system

Enable step-by-step solutions PRO

Question 1

+20

How many services are listening on the target system on all interfaces? (Not on localhost and IPv4 only)

> SSH to with user "**htb-student**" and password "**HTB_@cademy_stdnt!**"

Write your answer

Submit

Question 2

+20

Determine what user the ProFTPd server is running under. Submit the username as the answer.

Write your answer

Submit

Question 3 +1 XP +20

Use cURL from your Pwnbox (not the target machine) to obtain the source code of the "https://www.inlanefreight.com" website and filter all unique paths (https://www.inlanefreight.com/directory" or "/another/directory") of that domain. Submit the number of these paths as the answer.

Write your answer Submit